# Simrule: AI-Assisted Review of Auto Insurance Underwriting Decline Rules

Kazumi Li | AI Prototyping for Business Innovation (Ivey Online, delivered on Uplimit), Weeks 4-5 Innovation Challenge | October 15, 2026

**Live app:** https://jevwithwind.github.io/simrule/ **Code:** https://github.com/jevwithwind/simrule

*Academic prototype for Ivey Online coursework. Not affiliated with or representative of the Financial Services Regulatory Authority of Ontario (FSRA). All insurer submissions and precedent records are fictional.*

## 1. Executive Summary

Under Ontario's Take All Comers rule, an auto insurer may decline a consumer only on grounds the regulator has approved. Each proposed decline rule therefore controls access to insurance that drivers are legally required to hold. Review is document-heavy and judgment-heavy, and similar rules from different insurers can receive different outcomes.

**Simrule** is a working prototype of an AI-assisted review workbench, deployed as a static web app. It covers the path from submission to decision: document upload with reviewer-confirmed extraction; a compliance dashboard over 100 fictional submissions; a flagging system that assesses each rule against 15 criteria in the framework's four Levels, with issue codes, verbatim evidence and citations; a recommendation panel with similar rules, consumer impact and next steps; and a decision workflow with per-finding validation, override reasons, senior sign-off and an audit trail.

The central design decision is that the language model never sets the recommendation. It produces structured findings; a deterministic scoring function converts them into an advisory recommendation; a reviewer makes and owns the decision. Across the 100 test rules this gives 21 approve with conditions, 22 request more information and 57 recommend decline, each with reasoning on every criterion and explained precedents.

The expected impact is a faster first pass that starts from structured evidence, one standard applied across insurers, and a complete record of who decided what and why. The prototype does not replace legal or policy judgment, and its legal citations are not yet verified.

## 2. Methodology

I used Claude Code as the build environment instead of Bolt, Lovable or Replit, because the project needed a data pipeline, automated tests and version control alongside the interface. The optional chatbot and avatar video were not built.

| Component | Approach | Rationale |
|---|---|---|
| Web app | React, TypeScript and Vite; static build on GitHub Pages | No backend; state stays in the browser. |
| 100 assessments | Claude as a batch analyst under a versioned prompt; schema and content validation | Every rule gets a complete assessment that can be checked. |
| Recommendation | Deterministic scoring function over the findings | Identical findings give identical results. |
| Similar rules | TF-IDF cosine blended with structured feature overlap | Transparent and runs client-side; embeddings in production. |
| Architecture | Hand-laid SVG exported to PNG and PDF | One page showing human controls and the production path. |

### Approaches that worked

- **Schema-first prompting.** A schema plus content rules (all 15 criteria, 2 to 4 sentences of reasoning, at least one question for the insurer and two next steps). The validator, not inspection, decided when the prompt was ready.
- **A fixed issue-code taxonomy.** 25 codes (21 from the brief, 4 added) shared by the dashboard, consistency monitor and decision records.
- **Judgment separated from scoring.** Level 1 is a gate; Levels 2 to 4 are weighted 40, 35 and 25; thresholds are 75 and 50. All 15 weight and threshold variants tested matched the 13 anchor rules, so the starting values were frozen.
- **Evidence and citation controls.** All 940 evidence quotes are checked as exact substrings. Citations come only from a registry with a verified flag; official legal sites were unreachable from the build environment, so the 21 legal references stay at instrument level.

### Iteration

Prompt v1, piloted on 10 rules, passed validation once: the recommendations were already right, but reasoning collapsed to one-word findings and evidence was missing. I mapped 13 defects to 13 new rules (v2); the re-run passed 10 of 10 with no recommendation changing. In the full run, 9 of the remaining 90 failed first-pass validation, all for single-sentence reasoning. Similarity needed a second version: coarse trigger categories produced ties and false matches until I split them into 36 trigger types. A consistency audit then found 29 missing pending-submission flags, 9 explanations that did not say why similar rules diverged, and 2 misapplied findings; 38 edits across 29 assessments cleared them with no recommendation change. A 15-rule spot check agreed on 12, partly on 1 and disagreed on 2, which I left as override candidates rather than tuning findings. Finally, 134 unit tests, 6 end-to-end tests and an axe accessibility scan caught 4 defects that type-checking missed, including PDF upload failing in Chromium.

## 3. Limitations and Risks

### Gaps between prototype and production

- **Inference.** The 100 assessments are precomputed and frozen. New uploads get keyword triage only, unless a user supplies their own API key for the optional live assessment. Production needs server-side inference with retrieval over a versioned regulatory corpus, with cost and latency monitoring.
- **Security and records.** There is no authentication, roles are a demo switch, and data lives in browser storage. Confidential filings need access control, encryption and a retention policy.
- **Integration and validation.** There is no link to the regulator's filing systems, and the scoring weights were checked only against 13 anchor rules, not back-tested on historical decisions.
- **Coverage.** English only, no accessibility testing with real reviewers, and no ongoing monitoring of model quality.

### Risks before deployment

- **Citation errors.** Models can invent or misstate references. Citations are restricted to ids in a registry with a verified flag; the 21 legal references still need checking (VERIFY.md).
- **Automation bias.** Reviewers may accept findings without reading them. A decision is blocked until all 15 findings are validated, and the dashboard reports the acceptance rate and treats very high agreement as a warning.
- **Missed proxy discrimination.** Language, postal code or benefit status can track protected grounds. Keyword triage catches explicit wording only; disparate-impact testing and reviewer judgment are still required.
- **Gaming.** Insurers could draft rules around the issue codes while keeping the same effect. Reviewers must judge substance, and the taxonomy needs maintenance.
- **Correlated errors in evaluation.** The same AI drafted the assessments and ran the consistency audit, so its mistakes can be correlated. The 15-rule spot check is small and was done by the builder, not an independent reviewer.
- **Shallow text matching.** TF-IDF similarity misses rules that mean the same thing in different words, and keyword triage can misfire: in testing, "race" in "organized race" was flagged as a protected ground until the pattern was fixed.
- **Drift.** Models, guidance and markets change. Each assessment records its prompt and framework versions so it can be re-run, and the consistency monitor tracks overrides by issue code.
- **Privacy.** Filings may contain personal information. The optional live assessment sends text to an external API and must stay off for confidential material.

### Human oversight

Every step that can change an outcome needs a person. The reviewer confirms each extraction, then accepts or overrides all 15 findings, with a reason code and note for each override. Senior sign-off is required when the decision differs from the AI, a Level 1 finding is overridden, confidence is low, a Level 1 concern is raised or the case is escalated. The rationale is final only after the reviewer confirms "I have reviewed this rationale and take ownership of it". Every action is logged with time, actor, role, before and after values, and reason.

## 4. Personal Reflection

My background is quantitative finance: I model limit order book data and judge models by out-of-sample error. Building Simrule changed what I think AI is for in a regulated industry: its main value is not deciding faster, but making each judgment explicit enough for a person to check. The pilot showed this. The first prompt already reached the right recommendations on all 10 pilot rules, yet 9 of the 10 assessments failed validation because the reasoning and evidence were too thin to show a reviewer. For a regulator, a correct answer that cannot be explained has no value, so I now see transparency as the product rather than an add-on feature. Separating findings from a deterministic scoring rule is what made the system auditable: the same findings always give the same recommendation, and a reviewer can point to the exact finding they dispute.

I am a CEMS MIM student, currently on exchange in London. The project used the same toolkit a quant team relies on: a specified schema, a validation harness, thresholds calibrated against anchor cases, and an audit of outputs for consistency. I want to work in quantitative or AI roles in Japan, where I expect model governance to be at least as demanding as in this project. I plan to carry three habits from Simrule into that work: write the schema cautiously before the prompt, keep model judgment separate from the rule that acts on it, and test a model's explanations as rigorously as its accuracy. In trading research I would apply the same split, letting a model propose signals with evidence while transparent, versioned code decides how they are used, so a risk committee can audit both. I also want to design the audit trail first rather than last, because its accuracy and integrity matter most: it is the feature that makes everything else trustworthy.

What surprised me most was how often the checks I trusted were too weak to keep the AI in line. An automated consistency check passed 15 of 16 similar pairs with different outcomes, yet reading them showed that 8 explanations described the similarity without saying why the outcomes differed. The end-to-end test, not the unit tests, found that PDF upload failed in Chromium because a library called a JavaScript method the browser does not yet support. I was also surprised that the hardest problems were not technical: deciding how severe a missing actuarial study should be, or when a postal code becomes a proxy, needed judgment I could only write down after seeing many cases. The actuarial-severity rule had to be clarified at batch 7 because, read literally, it would have sent every new administrative rule to an information request. The consistency audit ended with 38 edits across 29 assessments and no change to any recommendation: the problems were in the explanations, not the scores. The lesson I take away is that trust in AI is earned by making disagreement easy: a reviewer must read the output line by line, or at least block by block, see the evidence, override it with a reason, and know that someone else will check.

# Appendix: Architecture and AI-Use Disclosure

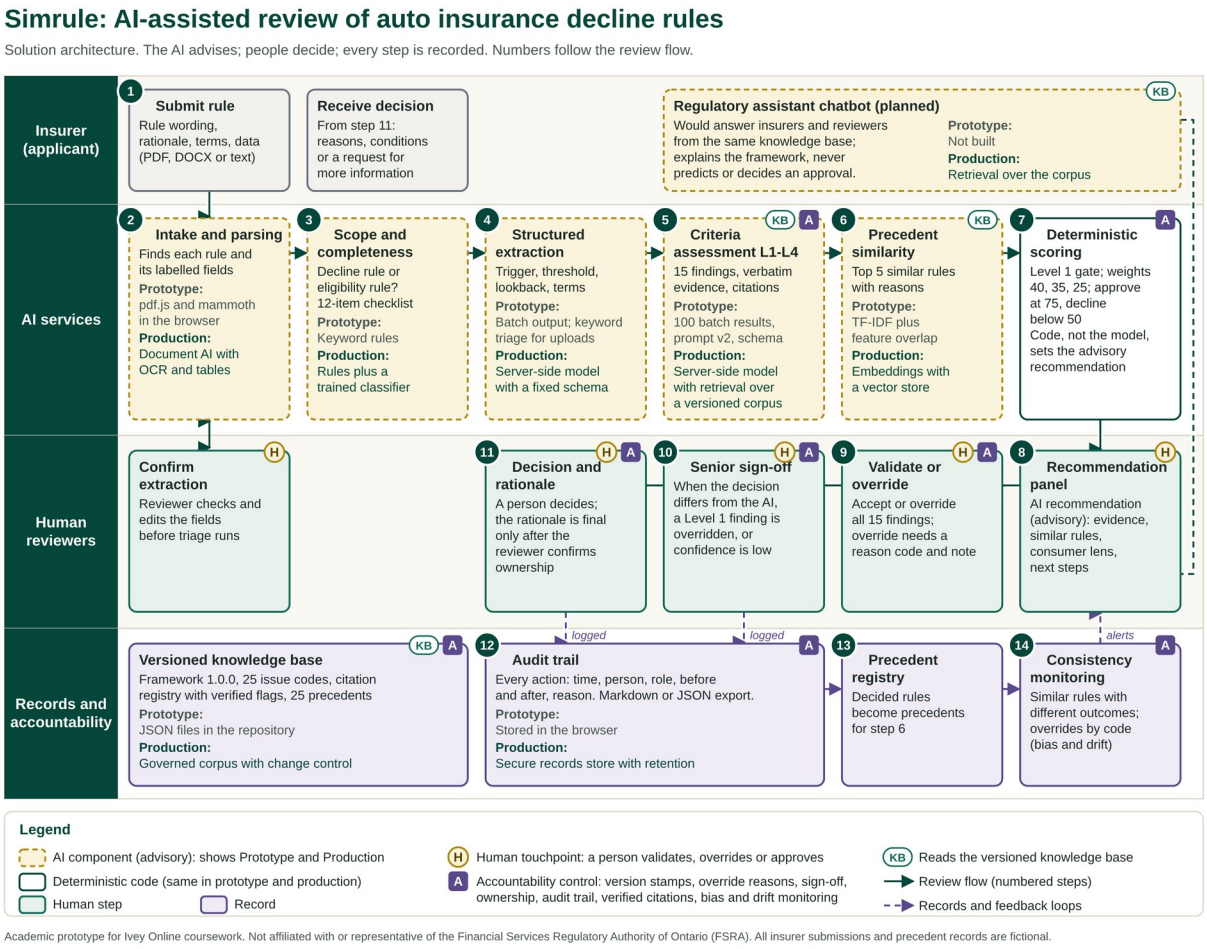



**Figure 1.** Solution architecture. Numbered steps follow one submission through four swimlanes; H marks human touchpoints and A marks accountability controls. Full size: deliverables/architecture/simrule-architecture.pdf and the How it works page of the live app.

## AI-use disclosure

Claude Code (Anthropic) was the engineering and analysis partner. Working from my written brief, it wrote the application code and tests, generated the 100 structured assessments under a versioned prompt (recommendations come from the deterministic scoring code, not the model), produced the architecture diagram and screen-recording script, and drafted this report from the project logs (PROGRESS.md and prompt-log.md). All figures come from those logs. Legal references were not verified against official sources; they are listed in VERIFY.md. I reviewed the outputs, made the final design decisions, and rewrote the personal reflection in my own words.